Použitie AI pri semestrálnej práci 1

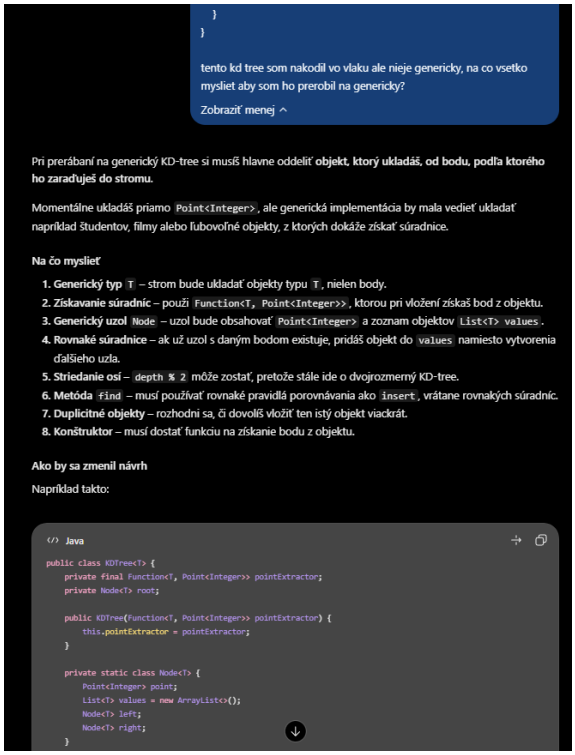
Pri práci som využíval nástroje umelej inteligencie **ChatGPT (model GPT-6)** a **GitHub Copilot Fusion – Inline Suggestions** v prostredí IntelliJ IDEA, pri ktorom sa model vyberá automaticky na základe úlohy.

Nástroj **Inline Suggestions** poskytoval priebežné návrhy kódu priamo v editore IntelliJ. Navrhnuté riešenia slúžili predovšetkým ako inšpirácia pri implementácii, nie ako hotové prevzaté riešenia.

AI som využíval najmä na konzultácie týkajúce sa priestorových stromov a adaptérov, riešenie problémov s načítavaním externých dátových štruktúr, vývoj POI Managera a mapovej časti aplikácie, ladenie aplikácie a úpravu LaTeX dokumentácie. Pomoc AI som využil aj pri tvorbe a úprave dokumentácie.

V nasledujúcej časti uvádzam vybrané snímky obrazovky z konverzácií, ktoré dokumentujú konkrétne prípady využitia umelej inteligencie počas vývoja.

Priestorové stromy



Konzultácia návrhu KDTree



Príklad uzla KDTree

pri rovnakych suradniciach, value vkladam do values

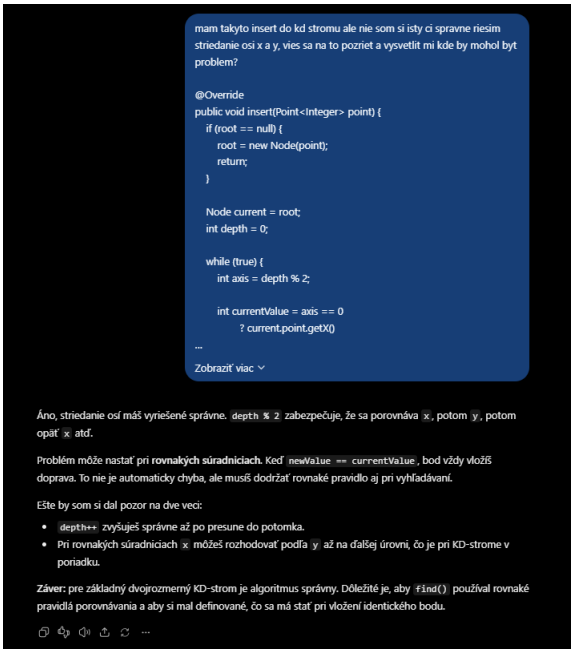
Aha, tak potom je to ešte jednoduchšie. Ak má každý uzol zoznam `values`, pri rovnakých súradniciach nemusíš vytvárať nový uzol.

Stačí pred porovnávaním osi skontrolovať, či sa súradnice zhodujú:

```java
if (current.point.equals(point)) {
    current.values.add(value);
    return;
}
```

Tým pádom bude mať každý uzol jedny súradnice a môže obsahovať viac objektov.

Ukladanie bodov s rovnakými súradnicami



Vkladanie a delenie priestoru

mam implementovany kd strom ktory momentalne pracuje priamo s Point Integer, ale potreboval by som ho upravit tak aby bol genericky a vedel ukladat aj ine objekty ktore maju nejake suradnice. ako by som to mal spravit aby som nemusel menit samotny princip fungovania stromu?

Najlepšie by bolo oddeliť **ukladaný objekt od jeho súradníc.** Namiesto toho, aby KD-strom pracoval priamo s `Point<Integer>`, môžeš použiť generický typ `T` a `Function<T, Point<Integer>>`, ktorá z objektu získa jeho súradnice.

Napríklad:

```java
public class KDTree<T> {
    private final Function<T, Point<Integer>> pointExtractor;

    public KDTree(Function<T, Point<Integer>> pointExtractor) {
        this.pointExtractor = pointExtractor;
    }
}
```

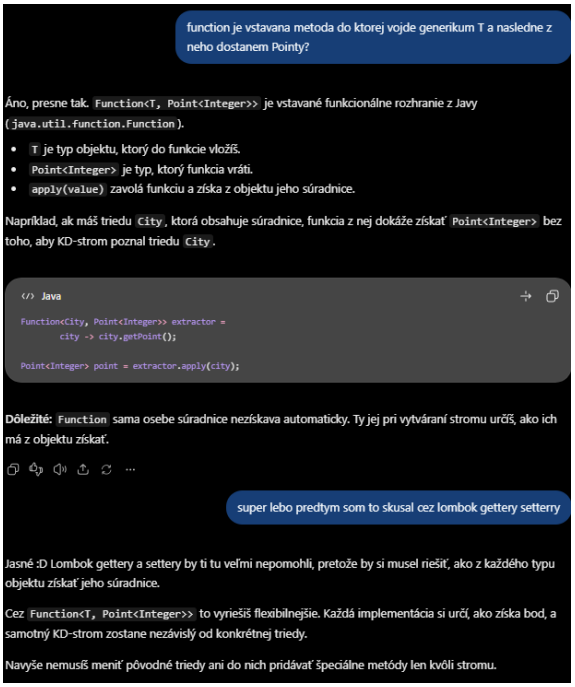
V uzloch by si potom ukladal objekty typu `T` a pri porovnávaní bodov používal `pointExtractor.apply(value)`.

Samotný algoritmus vkladania a vyhľadávania nemusíš meniť. Upravíš iba typy v uzloch a spôsob získavania súradníc. Takto budeš môcť ukladať napríklad `Point<Integer>` priamo, ale aj vlastné objekty, ktoré obsahujú súradnice.

Ak chceš generickosť aj pre typ súradníc, napríklad `Integer` alebo `Double`, bude potrebné navyše vyriešiť spôsob porovnávania súradníc.

function je vstavana metoda do ktorej vojde generikum T a nasledne z neho dostanem Pointy?

Oddelenie objektu od súradníc

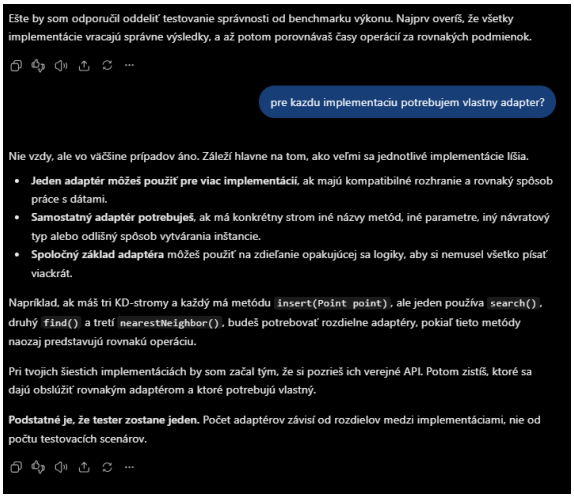


Získanie bodu z objektu

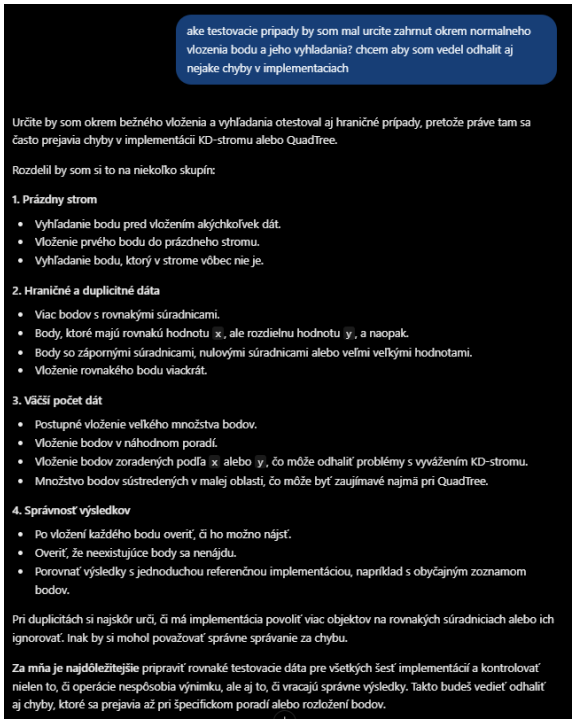
Adaptéry a testovanie



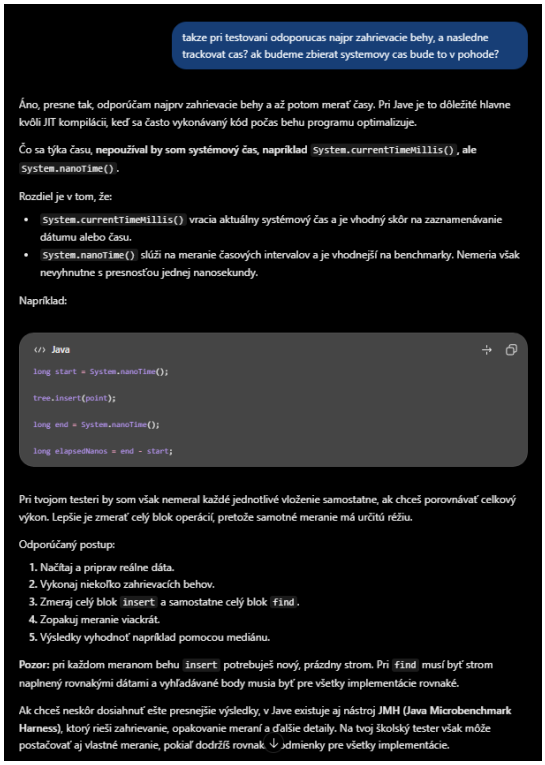
Spoločné rozhranie stromov



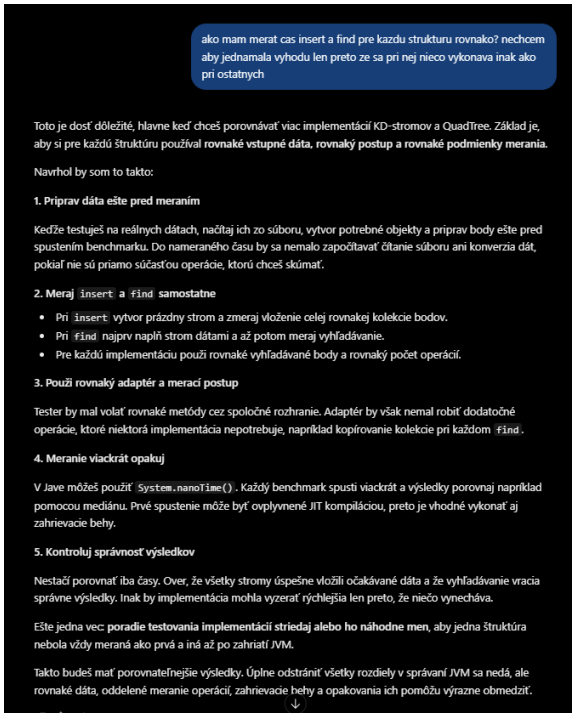
Adaptéry externých implementácií



Návrhy testovacích prípadov



Benchmark a zahrievacie behy

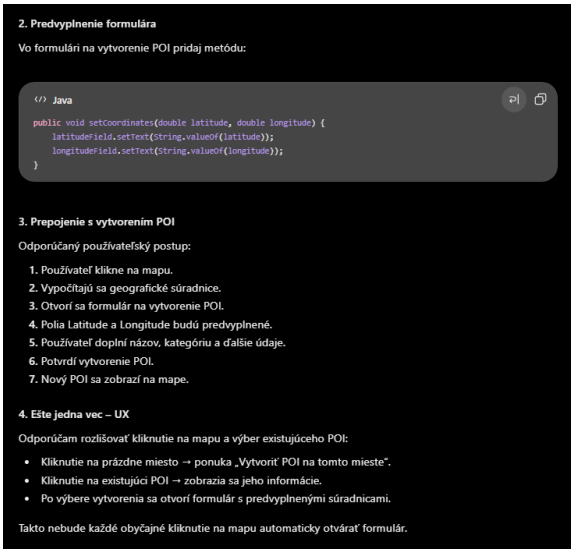


Porovnanie výkonu stromov

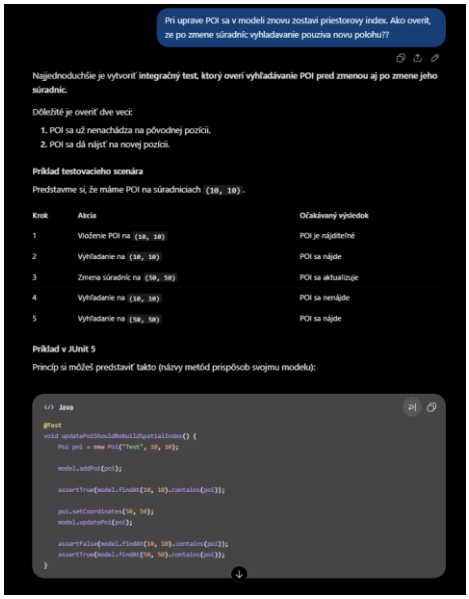
POI Manager3



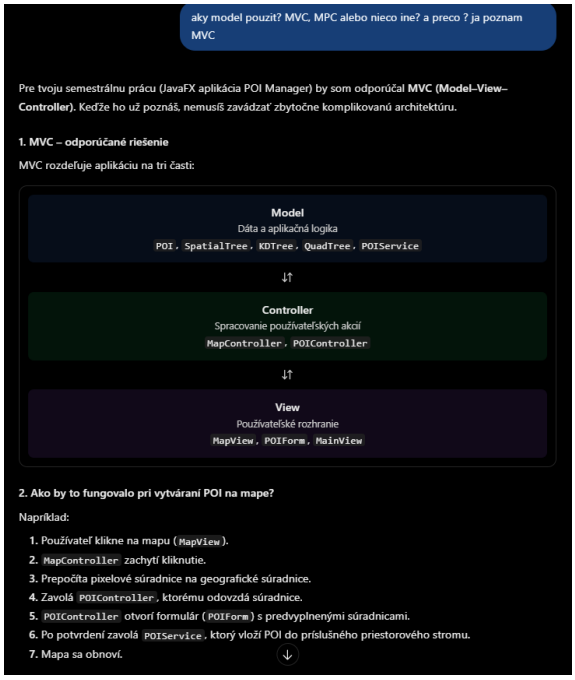
Mapa a vykreslenie bodov



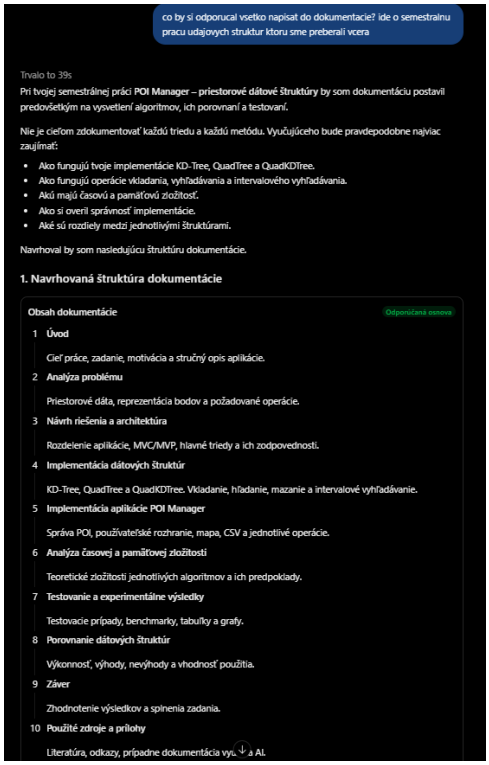
Formulár a vytvorenie POI



Úprava POI



Architektúra POI Managera



Ďalšia konzultácia k projektu

Rozhovory

ChatGPT – semestrálna práca

ChatGPT – opravy druhej časti